

Runtime Policy Enforcement at the AI Agent-to-Tool Boundary

Design and Validation of an Inline MCP/JSON-RPC Security Gateway

Evidence status: Refreshed against code snapshot 008775c8878cc70c50247baa223da3256e093316 and successful GitHub Actions CI run 37184196303 on 2026-09-30. The quantitative claims below are tied to that code snapshot. Do not generalize benchmark or test results beyond their stated scope.

723

Tests passed

Python 3.12 CI; Python 3.10/3.11 also green

0

Tests failed

Cited CI run

81.91%

Statement coverage

5,644 statements, 1,021 missed

55

Prompt-injection collection entries

Named runtime collection

69

Current merged detector (PR #123)

Current main at c30075d; separate from the 55-entry historical snapshot, 15/15 development attacks, 3/15 benign flags

9

Elastic rule records

detection_rules/elastic_rules.toml

21

Core SIEM tests

tests/test_siem.py

12

SIEM scenario tests

tests/test_siem_scenarios.py

Contribution

An application-layer inspection and authorization point for agent-generated MCP tools/call requests that combines JSON-RPC validation, normalization, trust/capability checks, heuristic content detection, policy evaluation, and tamper-evident audit telemetry before downstream tool execution.

Reproducibility

Expected at the cited snapshot: 723 passed, 81.91% statement coverage.

Honest Boundaries

- No external population-level false-positive/false-negative rate is established.
- Synthetic correlation fixtures and local microbenchmarks are not production efficacy or SLO evidence.
- Enforcement semantics are not identical across every integration path.
- Only traffic routed through the gateway can be inspected or blocked.
- A successful CI suite is not evidence of a live production SOC deployment.

Problem & Threat Model

LLM agents can turn untrusted natural-language context into structured tool calls with file, process, network, or privileged-service effects. The gateway protects the agent-to-tool boundary only when traffic is routed through it. Attackers may craft malicious tool arguments, malformed JSON-RPC, obfuscated prompt-injection strings, exfiltration payloads, or abusive request patterns.

Method

- Parse and validate JSON-RPC, including duplicate-key and malformed-input rejection.
- Normalize potentially obfuscated content before inspection.
- Evaluate server trust, capabilities, and content/policy signals.
- Return allow/block decisions using path-specific enforcement semantics.
- Record hash-chained audit evidence and SIEM-oriented telemetry.

Evidence Sources & References

- GitHub Actions CI run 37184196303
- VERIFIED_METRICS.md
- evidence/mcp_replay_evidence.json
- detection_rules/elastic_rules.toml
- tests/test_siem.py
- tests/test_siem_scenarios.py